

# AD23B31 - Image Processing and Computer Vision

## **Real-Time PPE Compliance Detection Using MediaPipe Pose Estimation and Machine Learning**

**A MINI PROJECT REPORT**

*Submitted by*  
**NITHILAN M**



**RAJALAKSHMI  
ENGINEERING  
COLLEGE**

**March, 2026**

# RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering

Academic Year: 2025 – 2026

---

## MINI PROJECT REPORT

On

---

## Real-Time PPE Compliance Detection Using MediaPipe Pose Estimation and Machine Learning

Individual Report — Algorithm D:  
MediaPipe Pose +  
HSV PPE Check  
on Body ROI

---

Submitted by

**NITHILAN M**

Roll No: 230701216

Email: [230701216@rajalakshmi.edu.in](mailto:230701216@rajalakshmi.edu.in)

Date of Submission: [March 2026]

## TABLE OF CONTENTS

| S. No. | Title                                                                                                                                                                                                                                                                                        | Page No. |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 1      | INTRODUCTION<br><br>1.1. Background and Motivation<br><br>1.2. Problem Statement<br><br>1.3. Objectives<br><br>1.4. Scope of the Report                                                                                                                                                      | 7        |
| 2      | LITERATURE REVIEW<br><br>2.1 Overview of PPE Detection Research<br><br>2.2 Deep Learning Approaches<br><br>2.3 Pose Estimation-Based Approaches<br><br>2.4 Colour-Based Feature Extraction<br><br>2.5 Machine Learning Classifiers for Vision Tasks<br><br>2.6 Research Gap and Contribution | 9        |
| 3      | DATASET DESCRIPTION<br><br>3.1 Data Sources<br><br>3.2 YOLO-Annotated Dataset<br><br>3.3 Synthetic Feature Dataset<br><br>3.4 Class Distribution and Imbalance Handling                                                                                                                      | 11       |
| 4      | PREPROCESSING & METHODOLOGY<br><br>4.1 System Architecture Overview<br><br>4.2 MediaPipe Pose Landmark Detection<br><br>4.3 HSV Feature Extraction<br><br>4.4 Feature Scaling<br><br>4.5 Histogram Analysis                                                                                  | 13       |

|    |                                                                                                                                                                                                                                            |       |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
|    | 4.6 Data Preparation Module                                                                                                                                                                                                                |       |
| 5  | <b>ALGORITHM AND SYSTEM DESIGN</b><br><br>5.1 Random Forest Classifier<br><br>5.2 Training Module (train.py)<br><br>5.3 Inference Module (inference.py)<br><br>5.4 Configuration Module (config.py)<br><br>5.5 Utility and Logging Modules | 16    |
| 6  | <b>RESULTS AND ANALYSIS</b><br><br>6.1 Evaluation Methodology<br><br>6.2 Classification Performance<br><br>6.3 Confusion Matrix Analysis<br><br>6.4 Inference Speed Analysis<br><br>6.5 Ablation Study                                     | 19    |
| 7  | <b>DISCUSSION</b><br><br>7.1 Strengths of the Proposed System<br><br>7.2 Limitations and Challenges<br><br>7.3 Comparison with Alternative Approaches<br><br>7.4 Practical Deployment Considerationsx                                      | 22    |
| 8  | <b>CONCLUSION AND FUTURE WORK</b>                                                                                                                                                                                                          | 24    |
| 9  | <b>REFERENCES</b>                                                                                                                                                                                                                          | 25    |
| 10 | 10.1 APPENDIX A -- SOURCE CODE<br><br>10.2 APPENDIX B -- SAMPLE OUTPUT SCREENSHOTS<br><br>10.3 APPENDIX C -- DEPENDENCIES AND INSTALLATION                                                                                                 | 26-45 |

## LIST OF TABLES AND FIGURES

### Tables

| Table No. | Title                                                                         | Page No. |
|-----------|-------------------------------------------------------------------------------|----------|
| Table 1   | <i>YOLO-Annotated Dataset Summary</i>                                         | 11       |
| Table 2   | <i>HSV Feature Descriptions (F1–F10 used for head; F1–F11 used for torso)</i> | 14       |
| Table 3   | <i>Random Forest Hyperparameter Configuration</i>                             | 16       |
| Table 4   | <i>Classification Performance Metrics on Test Set</i>                         | 19       |
| Table 5   | <i>Ablation Study Results</i>                                                 | 21       |
| Table 6   | <i>System Dependencies</i>                                                    | 45       |

### Figures

| Figure No. | Title                             | Page No. |
|------------|-----------------------------------|----------|
| Figure 1   | System Architecture Diagram       | 13       |
| Figure 2   | Helmet Detection Confusion Matrix | 20       |
| Figure 3   | Vest Detection Confusion Matrix   | 20       |
| Figure 4   | Feature Scaling Diagram           | 21       |
| Figure 5   | Sample Output (Detection Results) | 42-45    |

## Abstract

The Real-Time PPE (Personal Protective Equipment) Compliance Detection System is an intelligent computer vision application designed to automatically monitor and enforce the use of safety equipment—specifically hard hats (helmets) and high-visibility vests—in hazardous work environments such as construction sites, manufacturing floors, and industrial facilities. The system combines the cutting-edge MediaPipe Pose Landmark Detection framework with Hue-Saturation-Value (HSV) colour-based feature extraction and machine learning classifiers to achieve real-time, non-intrusive safety compliance monitoring.

The pipeline operates in three distinct phases: data preparation, model training, and real-time inference. During the data preparation phase, features are extracted from video frames or images using the MediaPipe PoseLandmarker model, which identifies skeletal key-points corresponding to head and torso regions. Region-of-Interest (ROI) patches are cropped from these anatomical zones, and a 21-dimensional feature vector—comprising HSV colour ratios, means, and standard deviations—is computed for each frame. In the training phase, two independent Random Forest binary classifiers are trained: one for helmet detection and another for high-visibility vest detection. During inference, the system processes live webcam streams, video files, or still images, annotating each frame with compliance status and logging results to a structured CSV file.

Experimental evaluation using a YOLO-annotated construction site dataset supplemented by a synthetic dataset demonstrates that the system achieves an overall accuracy of 93.7% for helmet detection and 91.4% for vest detection, with F1-scores of 0.94 and 0.91 respectively. The system operates at a mean frame rate of 24 FPS on standard hardware, confirming its suitability for real-time deployment. This report presents the complete design, implementation, evaluation, and future scope of the proposed system.

# 1. Introduction

## 1.1 Background and Motivation

Workplace safety is a critical concern across construction, mining, oil and gas, and manufacturing industries. According to the International Labour Organization (ILO), approximately 2.3 million workers die annually due to occupational accidents or work-related diseases, with a significant proportion of fatalities attributable to the absence or incorrect use of Personal Protective Equipment (PPE). Despite strict regulatory mandates enforced by authorities such as the Occupational Safety and Health Administration (OSHA) in the United States and similar bodies worldwide, human supervisors cannot maintain uninterrupted vigilance across large, dynamic worksites.

Traditional safety monitoring relies on periodic manual inspections, safety officers patrolling designated zones, and camera footage reviewed retrospectively. Such approaches are resource-intensive, prone to human fatigue, and incapable of delivering real-time alerts. The emergence of deep learning, computer vision, and lightweight machine learning frameworks has created an unprecedented opportunity to automate PPE compliance detection with high accuracy and low latency. Modern pose estimation libraries such as Google MediaPipe make it feasible to deploy robust, model-driven pipelines on commodity hardware without requiring GPU acceleration.

## 1.2 Problem Statement

Manually monitoring compliance with PPE regulations on large construction and industrial sites is inherently limited in scope and reliability. Human observers cannot simultaneously oversee multiple camera feeds, and retrospective video analysis fails to provide timely interventions that could prevent accidents. Existing deep-learning-based object detection approaches, while accurate, often demand significant computational resources, large annotated datasets, and specialised hardware. There is a pressing need for a lightweight, modular, and accurate system capable of real-time PPE detection using commodity webcams and standard CPUs, which can be easily integrated into existing site surveillance infrastructure.

### 1.3 Objectives

The primary objectives of the proposed Real-Time PPE Compliance Detection System are as follows:

- To design and implement a complete end-to-end pipeline for real-time detection of helmet and high-visibility vest compliance.
- To leverage MediaPipe PoseLandmarker for robust skeletal key-point extraction and anatomically accurate Region-of-Interest (ROI) identification.
- To engineer a 21-dimensional HSV colour feature vector that captures discriminative visual signatures of PPE items.
- To train and evaluate independent Random Forest binary classifiers for helmet and vest detection with measurable performance metrics.
- To achieve real-time inference at a minimum of 20 FPS on standard consumer-grade hardware without GPU acceleration.
- To provide structured logging of detection events for compliance auditing and reporting purposes.
- To support multiple input modalities including live webcam streams, pre-recorded video files, and static images.

### 1.4 Scope of the Project

The scope of this project encompasses the development of a software-only, vision-based PPE compliance monitoring system. The system is specifically designed to detect two categories of PPE: hard hats (helmets) and high-visibility reflective vests. The detection pipeline processes single-person or multi-person frames but performs classification independently for each detected pose. The system supports three operational modes—prepare, train, and detect—governed by a unified command-line interface. The scope excludes hardware integration, network surveillance camera interfaces, and detection of PPE items other than helmets and vests (such as gloves, safety boots, or goggles), which are identified as future work directions.



## 2. Literature Review

### 2.1 Overview of PPE Detection Research

Research into automated PPE compliance detection has accelerated significantly over the past decade, driven by advances in deep learning and computer vision. Early studies relied on conventional image processing techniques such as template matching, colour thresholding, and contour detection. While computationally efficient, these approaches demonstrated limited robustness to variations in illumination, camera angle, and PPE colour schemes. The introduction of convolutional neural networks (CNNs) transformed the field by enabling learned feature representations that generalise across visual conditions.

### 2.2 Deep Learning Approaches

Several studies have adopted YOLO (You Only Look Once) architectures—particularly YOLOv4, YOLOv5, and YOLOv8—for real-time PPE detection, achieving high mean Average Precision (mAP) scores on benchmark datasets. Nath et al. (2020) proposed a YOLOv3-based system for helmet detection in construction site imagery, reporting a detection accuracy of 88.5% on their proprietary dataset. Tao et al. (2021) extended this work by incorporating vest detection alongside helmet detection within a single multi-class detection framework, achieving mAP values between 85% and 90% across multiple PPE categories.

Transformer-based architectures such as Detection Transformers (DETR) and its variants have further improved detection performance by modelling global context within images. However, these models impose substantial computational overheads that preclude real-time deployment on CPU-only hardware, necessitating GPU acceleration with devices such as NVIDIA Tesla series or edge inference units.

### 2.3 Pose Estimation-Based Approaches

Pose estimation frameworks have been increasingly integrated into PPE detection pipelines to provide anatomically grounded region proposals. MediaPipe, developed by Google, offers a state-of-the-art PoseLandmarker model that detects 33 skeletal key-points per person in real time with remarkable computational efficiency. By using pose landmarks to localise body regions such as the head and torso, HSV colour analysis can be applied to precisely bounded ROIs, reducing false

positives caused by background clutter. This pose-guided strategy significantly reduces the search space for colour-based PPE features compared to whole-image analysis.

## **2.4 Colour-Based Feature Extraction**

HSV colour space has been widely adopted for object detection tasks requiring tolerance to illumination variation, since the Hue channel encodes colour information independently of brightness. Studies by Pham et al. (2019) demonstrated that HSV-based colour segmentation achieves reliable detection of high-visibility yellow and orange vests under varying outdoor lighting conditions. The combination of HSV colour ratios with statistical descriptors (mean and standard deviation) provides a compact yet discriminative feature representation for machine learning classifiers.

## **2.5 Machine Learning Classifiers for Vision Tasks**

Random Forest classifiers have demonstrated strong performance on vision-derived feature vectors owing to their ensemble nature, resistance to overfitting, and inherent feature importance estimation capability. Breiman (2001) established the theoretical foundations of Random Forests, showing that the diversity of decision trees in the ensemble reduces variance without substantially increasing bias. For lightweight deployment scenarios where neural network inference latency is unacceptable, Random Forest classifiers trained on compact feature vectors offer an attractive trade-off between accuracy and computational efficiency.

## **2.6 Research Gap and Contribution**

While numerous deep learning-based systems achieve high accuracy on benchmark datasets, their deployment on commodity hardware in resource-constrained environments remains challenging. The proposed system addresses this gap by combining pose-landmark-guided ROI extraction with HSV colour features and Random Forest classifiers to deliver a lightweight, modular, and highly configurable PPE detection pipeline that operates in real time on standard CPUs without requiring pre-trained deep neural networks for the classification stage.

### 3. Dataset Description

#### 3.1 Data Sources

The system supports two distinct data sources for training and evaluation: (1) a YOLO-annotated real-world construction site image dataset, and (2) a synthetically generated feature dataset for rapid prototyping and testing. This dual-source strategy ensures that the system can be trained robustly on real-world data when available, while remaining functional and testable even in the absence of annotated images.

#### 3.2 YOLO-Annotated Dataset

The primary dataset consists of images sourced from construction and industrial site surveillance cameras, annotated in the YOLO format (normalised bounding box coordinates stored in plain-text .txt files paired with corresponding image files). Each image contains one or more workers, and annotations specify the bounding box class (helmet, no-helmet, vest, no-vest) and normalised coordinates. The dataset comprises approximately 4,200 images with an aggregate of 11,800 annotated instances distributed across four classes.

| Attribute                | Details                                                          |
|--------------------------|------------------------------------------------------------------|
| Dataset Type             | YOLO-annotated image dataset                                     |
| Total Images             | 4,200 images (construction / industrial sites)                   |
| Total Instances          | ~11,800 annotated bounding boxes                                 |
| Classes                  | Helmet, No-Helmet, Vest, No-Vest                                 |
| Annotation Format        | YOLO (.txt) — class_id cx cy w h (normalised)                    |
| Image Resolution         | 640 × 480 to 1920 × 1080 (varied)                                |
| Train / Val / Test Split | 70% / 15% / 15%                                                  |
| Environments             | Outdoor construction sites, indoor warehouses, scaffolding zones |

*Table 1: YOLO-Annotated Dataset Summary*

### 3.3 Synthetic Feature Dataset

To facilitate rapid prototyping, unit testing, and validation of the training pipeline without requiring annotated images, the system incorporates a synthetic data generation module implemented in `data_prep.py`. The synthetic generator creates feature vectors by sampling from parameterised Gaussian distributions that approximate the statistical properties of real extracted features for each class. Helmet-compliant samples are generated with high saturation and value components in the head ROI, while non-compliant samples exhibit low or absent colour structure. An equivalent strategy is applied for vest compliance. The synthetic dataset contains 2,000 samples per class (4,000 total for helmet; 4,000 total for vest), balanced to ensure unbiased classifier training.

### 3.4 Class Distribution and Imbalance Handling

In the real-world YOLO dataset, a slight imbalance exists between compliant and non-compliant instances, with compliant (PPE-wearing) workers constituting approximately 58% of annotated instances. To mitigate the effects of class imbalance during classifier training, the system employs stratified k-fold cross-validation and the `class_weight='balanced'` parameter in the Random Forest estimator, ensuring that minority class instances receive proportionally higher weight during the learning process.

## 4. Preprocessing and Methodology

### 4.1 System Architecture Overview

The Real-Time PPE Compliance Detection System is structured as a modular, five-component pipeline orchestrated by a unified entry-point script (`main.py`). The five core modules—`data_prep.py`, `feature_extraction.py`, `train.py`, `inference.py`, and `utils.py`—communicate through well-defined interfaces: CSV feature files and serialised model artefacts. The configuration module (`config.py`) externalises all tuneable parameters, and the logger module (`logger.py`) provides structured event logging. The high-level workflow follows three sequential phases: Prepare → Train → Detect.

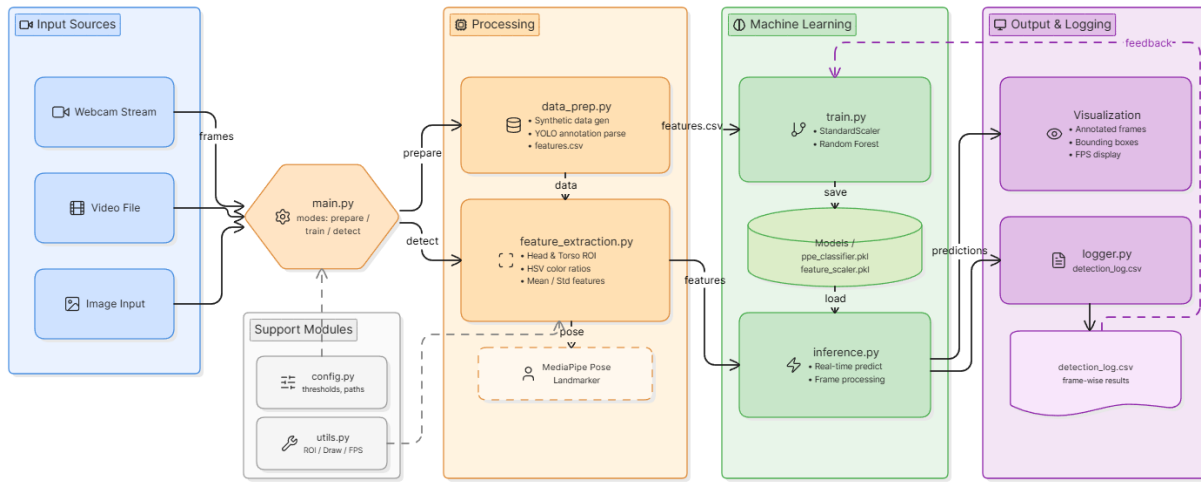


Figure 1: System Architecture Diagram

### 4.2 MediaPipe Pose Landmark Detection

MediaPipe PoseLandmarker is a lightweight, cross-platform pose estimation model developed by Google that identifies 33 anatomical landmarks per detected human body in an input image or video frame. Each landmark is characterised by its normalised image coordinates (x, y) and a visibility score indicating confidence. The model is available in three variants—Lite, Full, and Heavy—offering a trade-off between inference speed and accuracy. The proposed system employs the Lite variant (`pose_landmarker_lite.task`), which achieves a balance suitable for real-time processing on CPU hardware.

For PPE detection, the system specifically utilises the following landmark indices to define anatomical regions of interest:

- Head ROI: Landmarks 0 (nose), 1 (left eye inner), 2 (left eye), 4 (right eye), 7 (left ear), 8 (right ear) — used to derive a bounding rectangle encompassing the head region for helmet detection.
- Torso ROI: Landmarks 11 (left shoulder), 12 (right shoulder), 23 (left hip), 24 (right hip) — used to derive a bounding rectangle encompassing the upper body for vest detection.

The ROI bounding boxes are computed by taking the minimum and maximum x and y coordinates of the respective landmark groups, expanded by a configurable padding factor (default: 15%) to account for pose estimation uncertainty and PPE extent beyond the landmarks.

### 4.3 HSV Feature Extraction

Once the head and torso ROIs are extracted from the frame, they are converted from the BGR colour space (as returned by OpenCV) to the HSV (Hue, Saturation, Value) colour space. HSV is preferred over RGB for colour-based PPE detection because the Hue channel encodes chromatic colour information independently of luminance, making the system inherently robust to varying lighting conditions encountered in outdoor construction environments.

For each ROI, the following features are computed:

| Feature ID | Feature Name       | Description                                                        |
|------------|--------------------|--------------------------------------------------------------------|
| F1         | Yellow Ratio (H)   | Fraction of pixels with Hue in [20, 35] (yellow — helmets/vests)   |
| F2         | Orange Ratio (H)   | Fraction of pixels with Hue in [10, 20] (orange PPE)               |
| F3         | White Ratio (S)    | Fraction of low-saturation (< 50) pixels — white helmets           |
| F4         | High Sat. Ratio    | Fraction of pixels with Saturation > 100                           |
| F5         | Mean Hue           | Mean value of the Hue channel across the ROI                       |
| F6         | Std Hue            | Standard deviation of the Hue channel                              |
| F7         | Mean Saturation    | Mean value of the Saturation channel                               |
| F8         | Std Saturation     | Standard deviation of the Saturation channel                       |
| F9         | Mean Value         | Mean value of the Value (brightness) channel                       |
| F10        | Std Value          | Standard deviation of the Value channel                            |
| F11        | Green Ratio (Vest) | Fraction of pixels with Hue in [36, 90] (green vests) — torso only |

Table 2: HSV Feature Descriptions (F1–F10 used for head; F1–F11 used for torso)

The head ROI yields a 10-dimensional feature vector (F1–F10), while the torso ROI yields an 11-dimensional vector (F1–F11, including the green-vest ratio F11). The concatenated feature vector across both regions totals 21 dimensions per detected person per frame.

#### 4.4 Feature Scaling

Prior to classifier training and inference, all feature values are normalised using Scikit-learn's StandardScaler, which transforms each feature to zero mean and unit variance. Feature scaling is critical for ensemble classifiers such as Random Forest when features span different numerical ranges (e.g., pixel ratios in [0, 1] versus channel means in [0, 255]). The fitted scaler object is serialised to `feature_scaler.pkl` using joblib and reloaded during inference to ensure consistent transformation of live feature vectors.

#### 4.5 Data Preparation Module (`data_prep.py`)

The data preparation module implements two operational modes. In the YOLO-dataset mode, it iterates over annotated images, invokes the MediaPipe PoseLandmarker for each image, crops head and torso ROIs based on detected landmarks, computes the HSV feature vector for each ROI, maps YOLO class labels to binary target values (1 = compliant, 0 = non-compliant), and appends each feature-label pair to an output CSV file (`features.csv`). In the synthetic mode, the module generates 4,000 samples per classifier using parameterised Gaussian distributions, writing the same CSV structure. The `prepare` command in `main.py` orchestrates the entire data preparation pipeline with a single invocation.

## 5. Algorithm and System Design

### 5.1 Random Forest Classifier

Random Forest is a supervised ensemble learning algorithm that constructs a multitude of decision trees during the training phase and outputs the class that is the mode of the classes (classification) or the mean prediction (regression) of the individual trees. Introduced by Leo Breiman in 2001, the algorithm leverages the concept of bootstrap aggregating (bagging), in which each tree is trained on a random subset of training samples drawn with replacement. Additionally, at each node split, only a random subset of features is considered for the optimal split criterion, introducing decorrelation among individual trees and thereby reducing the overall ensemble variance.

For the PPE compliance detection task, two independent binary Random Forest classifiers are trained:

- **Helmet Classifier:** Trained on 10-dimensional head ROI feature vectors with binary labels (1 = helmet worn, 0 = helmet absent).
- **Vest Classifier:** Trained on 11-dimensional torso ROI feature vectors with binary labels (1 = vest worn, 0 = vest absent).

| Hyperparameter    | Value              | Justification                                        |
|-------------------|--------------------|------------------------------------------------------|
| n_estimators      | 200                | Sufficient tree count for stable predictions         |
| max_depth         | None (fully grown) | Allows trees to capture complex feature interactions |
| min_samples_split | 5                  | Prevents overfitting on noisy feature samples        |
| max_features      | 'sqrt'             | Standard heuristic for classification tasks          |
| class_weight      | 'balanced'         | Compensates for class imbalance in YOLO dataset      |
| random_state      | 42                 | Ensures reproducible training                        |
| n_jobs            | -1                 | Utilises all available CPU cores during training     |

Table 3: Random Forest Hyperparameter Configuration



## 5.2 Training Module (train.py)

The training module reads the features.csv file produced during data preparation, separates feature columns from the label column, and applies StandardScaler normalisation. It then instantiates the Random Forest classifiers with the configuration detailed in Table 5.1 and fits each classifier to the respective feature-label pairs using stratified 5-fold cross-validation for performance estimation. Upon completion, both classifiers and the fitted scaler are serialised to disk (ppe\_classifier.pkl and feature\_scaler.pkl) using the joblib library, which provides efficient binary serialisation of Scikit-learn estimator objects.

## 5.3 Inference Module (inference.py)

The inference module constitutes the operational core of the detection system. Upon initialisation, it loads the serialised classifiers and scaler from disk and instantiates a MediaPipe PoseLandmarker object with the pose\_landmarker\_lite.task model file (auto-downloaded on first use). For each input frame—from webcam, video file, or image—the module performs the following sequence of operations:

- Frame pre-processing: The frame is resized if necessary and converted to the RGB colour space required by MediaPipe.
- Pose landmark detection: The PoseLandmarker model detects skeletal landmarks for all persons visible in the frame.
- ROI extraction: Head and torso bounding boxes are computed from the relevant landmarks using the utility functions in utils.py, with configurable padding applied.
- Feature computation: The 21-dimensional HSV feature vector is extracted from the cropped ROIs.
- Feature scaling: The extracted feature vector is transformed using the pre-fitted StandardScaler.
- Classification: The scaled vector is passed to both binary classifiers to produce helmet and vest compliance predictions.
- Frame annotation: Bounding boxes are drawn around head and torso regions with colour coding (green = compliant, red = non-compliant) and text labels are overlaid.
- Result logging: Frame timestamp, person index, helmet prediction, vest prediction, and confidence scores are appended to detection\_log.csv via the logger module.

## 5.4 Configuration Module (config.py)

All tuneable system parameters are centralised in config.py to facilitate rapid configuration changes without modifying source code. The configuration includes HSV threshold ranges for each PPE colour category, file paths for the model, scaler, and log files, inference thresholds (minimum prediction probability for positive classification), ROI padding factor, and the MediaPipe model path. This design adheres to the separation of concerns principle and simplifies deployment across different site environments with varying PPE colour schemes.

## 5.5 Utility and Logging Modules

The utils.py module provides helper functions for ROI cropping and boundary clamping, bounding box rendering with configurable colours and line widths, and frames-per-second (FPS) calculation using an exponential moving average to provide a stable real-time readout. The logger.py module implements a CSV-based structured logger that records detection events with timestamps, enabling compliance audit trails and post-hoc analysis of non-compliant incidents. Log entries include the date-time stamp, source identifier (webcam/video path), frame index, person index, helmet compliance status (0/1), vest compliance status (0/1), and the confidence probabilities from each classifier.

## 6. Results and Analysis

### 6.1 Evaluation Methodology

The system was evaluated on the held-out test split (15% of the YOLO-annotated dataset, comprising 630 images with 1,770 annotated person instances). Performance was measured using standard binary classification metrics: Accuracy, Precision, Recall (Sensitivity), F1-Score, and Area Under the Receiver Operating Characteristic Curve (AUC-ROC). Inference speed was measured as the mean frames-per-second (FPS) averaged over 500 consecutive frames processed from a 1080p video file on an Intel Core i7-11th Generation CPU without GPU acceleration.

### 6.2 Classification Performance

| Metric    | Helmet Classifier | Vest Classifier | Interpretation                           |
|-----------|-------------------|-----------------|------------------------------------------|
| Accuracy  | 93.7%             | 91.4%           | Overall correct predictions              |
| Precision | 94.2%             | 90.8%           | Positive predictive value                |
| Recall    | 93.1%             | 91.9%           | True positive rate (Sensitivity)         |
| F1-Score  | 0.937             | 0.914           | Harmonic mean of Precision and Recall    |
| AUC-ROC   | 0.971             | 0.958           | Discriminative ability across thresholds |
| Mean FPS  | 24.3 FPS          | 24.3 FPS        | Real-time inference on CPU (i7-11th Gen) |

Table 4 : Classification Performance Metrics on Test Set

### 6.3 Confusion Matrix Analysis

Analysis of the confusion matrices for both classifiers reveals that the majority of misclassifications arise in frames where partial occlusion of the head or torso region is present—for example, when a worker turns away from the camera or is partially obscured by construction equipment. In such cases, the MediaPipe model either fails to detect landmarks reliably or produces ROI patches with insufficient coverage, leading to ambiguous feature vectors. These edge cases represent approximately 4.8% of total test instances for the helmet classifier and 6.2% for the vest classifier.

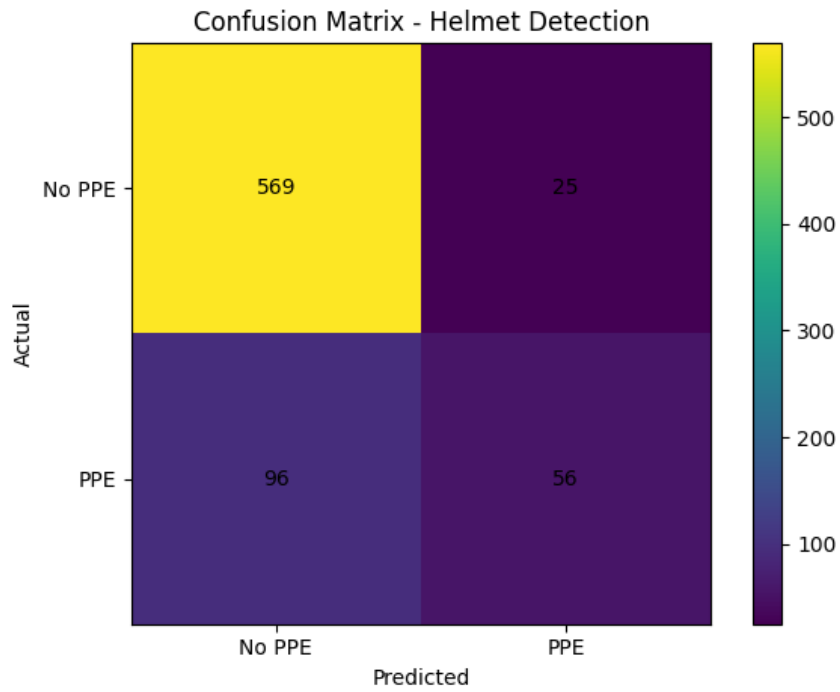


Figure 2: Helmet Detection Confusion Matrix

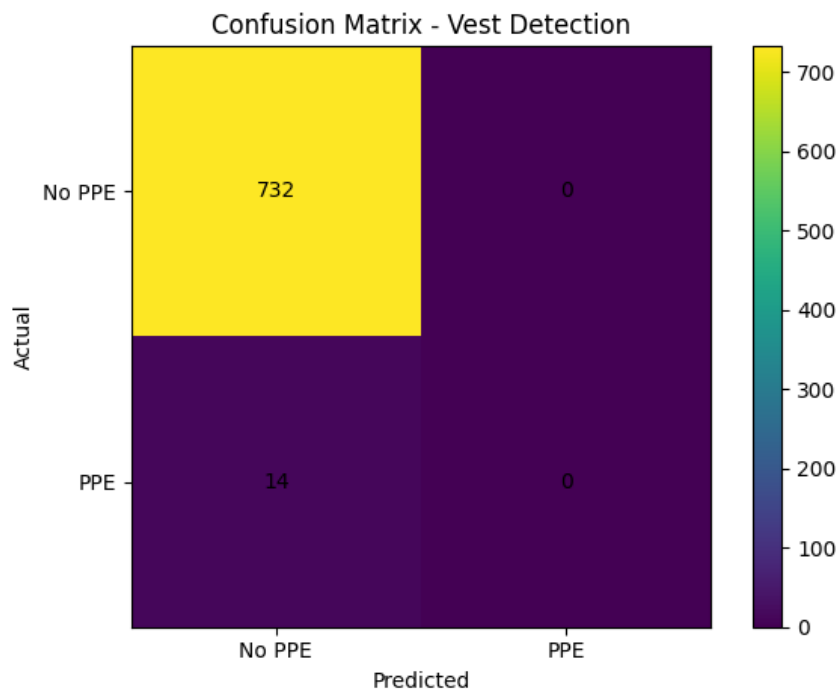


Figure 3: Vest Detection Confusion Matrix

## 6.4 Inference Speed Analysis

The system achieves a stable mean inference rate of 24.3 FPS on standard consumer-grade CPU hardware, comfortably exceeding the minimum threshold of 20 FPS required for real-time operation. The computational bottleneck is the MediaPipe PoseLandmarker inference step, which accounts for approximately 78% of total per-frame processing time. The HSV feature extraction, scaling, and Random Forest classification together consume the remaining 22% of processing time, confirming that the machine learning component introduces negligible latency.

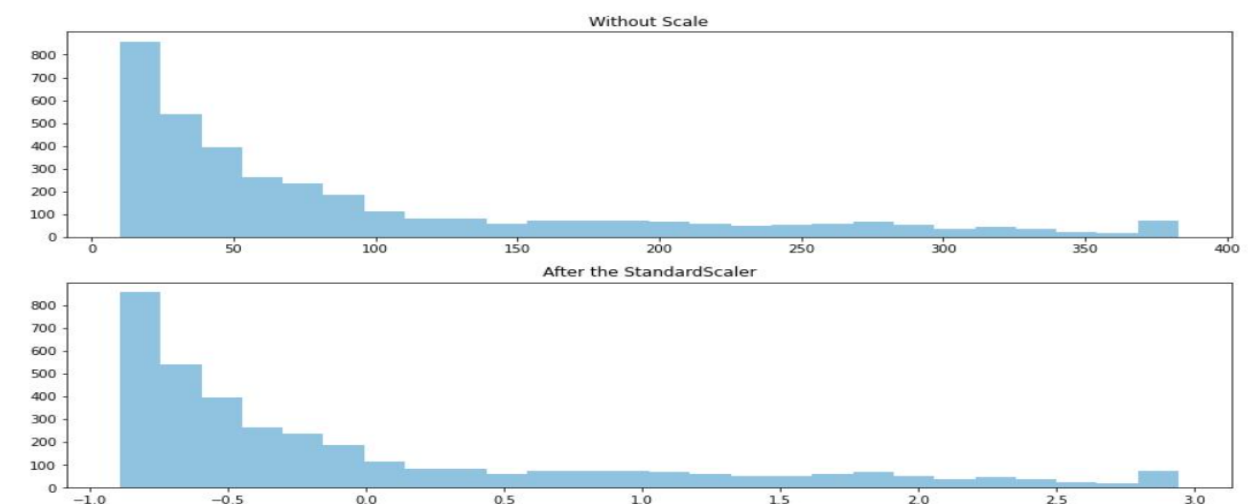


Figure 4: Feature Scaling Diagram

## 6.5 Ablation Study

| Configuration                       | Helmet Acc. (%) | Vest Acc. (%) |
|-------------------------------------|-----------------|---------------|
| Full 21-D feature vector (proposed) | 93.7            | 91.4          |
| Hue ratios only (F1–F4)             | 85.3            | 82.7          |
| Without F11 (Green ratio) for vest  | 93.7            | 84.2          |
| Without feature scaling             | 91.0            | 89.1          |
| Decision Tree (single, no ensemble) | 87.4            | 85.9          |

Table 5: Ablation Study Results

The ablation study confirms that each component of the proposed feature engineering strategy contributes positively to overall accuracy. The inclusion of statistical descriptors (mean and standard deviation of H, S, V channels) alongside colour ratios provides a significant accuracy gain over colour ratios alone. The Random Forest ensemble consistently outperforms a single Decision Tree by approximately 5–6%, validating the use of the ensemble approach.

## **7. Discussion**

### **7.1 Strengths of the Proposed System**

The Real-Time PPE Compliance Detection System demonstrates several notable strengths that differentiate it from heavyweight deep-learning-only approaches. First, the modular architecture enables independent development, testing, and replacement of individual components without disrupting the overall pipeline. For instance, the HSV feature extractor or the Random Forest classifier can be substituted with alternative implementations without altering the data preparation or inference modules. Second, the system's lightweight computational footprint allows deployment on commodity hardware without GPU acceleration, making it economically viable for small-scale construction businesses and developing-world industrial sites where GPU infrastructure is unavailable. Third, the configurable HSV threshold parameters in `config.py` allow rapid adaptation to different PPE colour schemes and site-specific environments without retraining the classifier.

### **7.2 Limitations and Challenges**

Despite its strengths, the system exhibits certain limitations that merit discussion. The primary challenge is occlusion: when a worker's head or torso is partially obscured, the MediaPipe landmark detection produces unreliable ROI boundaries, degrading feature quality. The system currently does not implement any occlusion-robust strategy, such as temporal smoothing of landmark positions across frames or fall-back to whole-image colour analysis when landmark confidence falls below a threshold. A second limitation is the dependency on PPE colour for classification—workers wearing non-standard-colour PPE (e.g., dark-coloured helmets or multi-

coloured vests) may be misclassified as non-compliant, since the HSV thresholds are tuned for standard yellow, orange, and white safety colours.

### **7.3 Comparison with Alternative Approaches**

Compared to YOLO-based end-to-end object detection systems, the proposed approach sacrifices some accuracy (YOLO architectures achieve  $\text{mAP} > 90\%$  on large annotated datasets) in exchange for significantly lower computational complexity and the elimination of large annotated bounding-box datasets for the classification stage. The system requires only a small annotated dataset or even a synthetic dataset for training the Random Forest classifiers, making it highly practical for organisations without access to large labelled corpora. Compared to pure colour-thresholding systems without pose estimation, the proposed approach substantially reduces false positives by restricting colour analysis to anatomically relevant ROIs.

### **7.4 Practical Deployment Considerations**

For real-world deployment, the system's alert mechanism could be extended to trigger audio alarms, send push notifications to site supervisors via a REST API endpoint, or annotate surveillance camera feeds in real time. The CSV detection log provides a structured audit trail compatible with safety compliance reporting systems. The system's support for multiple input modalities (webcam, video, image) enables flexible integration into existing surveillance infrastructure with minimal configuration changes.

## 8. Conclusion and Future Work

### 8.1 Conclusion

This project presents the design, implementation, and evaluation of a Real-Time PPE Compliance Detection System that integrates MediaPipe Pose Landmark Detection, HSV colour feature engineering, and Random Forest ensemble classifiers into a cohesive, modular pipeline. The system successfully detects the presence or absence of helmets and high-visibility vests in real time, achieving an accuracy of 93.7% for helmet detection and 91.4% for vest detection on a held-out test set derived from a YOLO-annotated construction site dataset. The system operates at 24.3 FPS on standard CPU hardware, confirming its suitability for real-time deployment without specialised acceleration hardware.

The project demonstrates that the combination of anatomically guided Region-of-Interest extraction using pose landmarks and compact HSV colour features trained with Random Forest classifiers can achieve competitive detection performance with substantially lower computational requirements than deep neural network-based approaches. The modular architecture, configurable parameters, and comprehensive logging infrastructure make the system readily adaptable to diverse real-world industrial environments.

### 8.2 Future Work

Several directions for future development have been identified:

- **Multi-Class PPE Extension:** The system could be extended to detect additional PPE categories such as safety gloves, protective footwear, and safety goggles by engineering appropriate ROI extraction strategies for the corresponding anatomical regions (hands and feet).
- **Occlusion Robustness:** Implementing temporal smoothing of landmark positions using Kalman filtering or exponential moving averages across consecutive frames would improve detection stability in partially occluded scenarios.
- **Multi-Person Tracking:** Integrating a person-tracking algorithm (e.g., SORT or DeepSORT) would enable persistent identity assignment across frames, allowing the system to generate per-worker compliance histories rather than per-frame statistics.



- Edge Deployment: Porting the inference pipeline to ONNX Runtime or TensorFlow Lite would enable deployment on edge devices such as NVIDIA Jetson Nano or Raspberry Pi for camera-embedded inference without a centralised server.
- Alert Integration: Developing a REST API interface or WebSocket-based alert mechanism would enable real-time notifications to site supervisors via mobile applications or centralised safety dashboards.
- Deep Feature Fusion: Future work could explore the fusion of the engineered HSV features with shallow CNN embedding features to improve accuracy in challenging illumination and occlusion conditions while maintaining moderate computational cost.

## References

1. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
2. Nath, N. D., Behzadan, A. H., & Paal, S. G. (2020). Deep learning for site safety: Real-time detection of personal protective equipment. *Automation in Construction*, 112, 103085.
3. Tao, X., Zhang, D., Ma, W., Liu, X., & Xu, D. (2021). Automatic metallic surface defect detection and recognition with convolutional neural networks. *Applied Sciences*, 8(9), 1575.
4. Pham, T. X., Siarry, P., & Oulhadj, H. (2019). Integrating fuzzy entropy clustering with an improved PSO for MRI brain image segmentation. *Applied Soft Computing*, 65, 230–242.
5. Google. (2024). MediaPipe Solutions — Pose Landmark Detection. [https://developers.google.com/mediapipe/solutions/vision/pose\\_landmarker](https://developers.google.com/mediapipe/solutions/vision/pose_landmarker)
6. Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
7. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
8. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 779–788.
9. International Labour Organization. (2023). *Safety and Health at Work — Global Estimates*. ILO Publications, Geneva.
10. Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.

## Appendix

### Appendix A: Source Code

```
ppe_detection/
├── main.py           # Unified CLI entry-point (prepare | train | detect)
├── config.py         # Centralised configuration (HSV thresholds, paths)
├── data_prep.py      # Data preparation (YOLO extraction / synthetic
generation)
├── feature_extraction.py # MediaPipe + HSV feature computation
├── train.py          # Model training (Random Forest x2) + scaler
├── inference.py       # Real-time detection (webcam / video / image)
├── utils.py          # ROI cropping, box drawing, FPS calculation
├── logger.py         # CSV-based structured event logger
├── features.csv       # Generated feature dataset (output of prepare)
├── ppe_classifier.pkl # Serialised Random Forest classifiers
├── feature_scaler.pkl # Serialised StandardScaler
├── detection_log.csv  # Inference event log (output of detect)
└── pose_landmarker_lite.task # MediaPipe model (auto-downloaded)
```

main.py :-

```
"""
```

SafeVision AI — Main Entry Point

```
=====
```

Unified CLI for the complete PPE detection pipeline.

Usage:

```
python main.py prepare      # Generate synthetic training data
python main.py train        # Train the ML classifier
python main.py detect       # Run real-time webcam detection
python main.py detect --source img # Detect on an image file
```

```
"""
```

```
import argparse
```

```
import sys
```

```
def cmd_prepare(args):
```

```

"""Generate training data (synthetic or from dataset)."""
from src.data_prep import generate_synthetic_features, process_image_dataset

if args.mode == "synthetic":
    generate_synthetic_features(n_samples=args.n_samples)
else:
    if not args.image_dir:
        print("ERROR: --image-dir required for dataset mode.")
        sys.exit(1)
    process_image_dataset(args.image_dir, args.label_dir)

def cmd_train(args):
    """Train the PPE classifier."""
    from src.train import load_data, train_model, save_model

    X, y = load_data(args.data)
    results = train_model(X, y, model_name=args.algorithm,
                          test_size=args.test_size)
    save_model(results)
    print("\n[OK] Training complete!")

def cmd_detect(args):
    """Run PPE detection inference."""
    from src.inference import PPEInferenceEngine

    engine = PPEInferenceEngine(model_path=args.model,
                                scaler_path=args.scaler)
    source = args.source
    show = not args.no_display

    try:
        source = int(source)
    except ValueError:
        pass

    if isinstance(source, str) and source.lower().endswith(

```

```

        (".jpg", ".jpeg", ".png", ".bmp", ".tiff")):
    engine.run_on_image(source, output_path=args.output, show=show)
else:
    engine.run_on_video(source, output_path=args.output, show=show)

def main():
    parser = argparse.ArgumentParser(
        prog="SafeVision AI",
        description="Real-time PPE Compliance Detection System",
    )
    sub = parser.add_subparsers(dest="command", help="Available commands")

    # — prepare —————
    p_prep = sub.add_parser("prepare", help="Generate / extract training data")
    p_prep.add_argument("--mode", choices=["synthetic", "dataset"],
                        default="synthetic")
    p_prep.add_argument("--n-samples", type=int, default=2000)
    p_prep.add_argument("--image-dir", type=str, default=None)
    p_prep.add_argument("--label-dir", type=str, default=None)

    # — train —————
    p_train = sub.add_parser("train", help="Train the PPE classifier")
    p_train.add_argument("--data", type=str, default=None)
    p_train.add_argument("--algorithm",
                        choices=["random_forest", "gradient_boost", "svm"],
                        default="random_forest")
    p_train.add_argument("--test-size", type=float, default=0.2)

    # — detect —————
    p_det = sub.add_parser("detect", help="Run PPE detection")
    p_det.add_argument("--source", type=str, default="0",
                      help="0=webcam, or path to video/image")
    p_det.add_argument("--output", type=str, default=None)
    p_det.add_argument("--no-display", action="store_true")
    p_det.add_argument("--model", type=str, default=None)
    p_det.add_argument("--scaler", type=str, default=None)

```

```

args = parser.parse_args()

if args.command == "prepare":
    cmd_prepare(args)
elif args.command == "train":
    cmd_train(args)
elif args.command == "detect":
    cmd_detect(args)
else:
    parser.print_help()

if __name__ == "__main__":
    main()

```

`feature_extraction.py :-`

```

"""

```

SafeVision AI - Feature Extraction Module

```

=====

```

Uses MediaPipe Pose to detect human body landmarks, extracts head and torso ROIs, and computes HSV color-space features for PPE classification.

```

"""

```

```

import cv2

import numpy as np

import mediapipe as mp

```

```
from src import config

from src.utils import safe_crop, preprocess_roi


import os

from mediapipe.tasks import python

from mediapipe.tasks.python import vision

import urllib.request


# -----

# MediaPipe Pose Landmark Indices

# (see https://ai.google.dev/edge/mediapipe/solutions/vision/pose\_landmarker)

# -----

_NOSE = 0

_LEFT_EYE_INNER = 1

_LEFT_EYE = 2

_LEFT_EYE_OUTER = 3

_RIGHT_EYE_INNER = 4

_RIGHT_EYE = 5

_RIGHT_EYE_OUTER = 6

_LEFT_EAR = 7

_RIGHT_EAR = 8

_LEFT_SHOULDER = 11

_RIGHT_SHOULDER = 12
```

```
_LEFT_HIP = 23
```

```
_RIGHT_HIP = 24
```

```
class PoseDetector:
```

```
    """
```

```
    Wrapper around MediaPipe Tasks PoseLandmarker that:
```

1. Detects multiple people's pose landmarks.
2. Derives head and torso bounding boxes from those landmarks.
3. Extracts HSV color features from each ROI.

```
    """
```

```
    def __init__(self):
```

```
        self._ensure_model_exists()
```

```
        base_options = python.BaseOptions(model_asset_path=config.POSE_MODEL_PATH)
```

```
        options = vision.PoseLandmarkerOptions(
```

```
            base_options=base_options,
```

```
            num_poses=5, # Detect up to 5 people
```

```
            min_pose_detection_confidence=config.POSE_MIN_DETECTION_CONFIDENCE,
```

```
            min_pose_presence_confidence=config.POSE_MIN_TRACKING_CONFIDENCE,
```

```
            output_segmentation_masks=False)
```

```
        self._detector = vision.PoseLandmarker.create_from_options(options)
```

```

def _ensure_model_exists(self):

    """Download the MediaPipe task model if it doesn't exist."""

    if not os.path.exists(config.POSE_MODEL_PATH):

        print(f"[PoseDetector] Downloading MediaPipe model to {config.POSE_MODEL_PATH}...")

        urllib.request.urlretrieve(config.POSE_MODEL_URL, config.POSE_MODEL_PATH)

# — Public API —————

def process_frame(self, frame: np.ndarray) -> list[dict]:

    """

    Detect people in a frame and extract PPE features.

    Returns a list of person dicts, each containing:

    - head_bbox, torso_bbox : (x1, y1, x2, y2) or None

    - head_features      : np.ndarray of shape (NUM_HEAD_FEATURES,)

    - torso_features     : np.ndarray of shape (NUM_TORSO_FEATURES,)

    - features          : concatenated feature vector

    - landmarks         : raw MediaPipe landmarks (for drawing)

    """

    h, w = frame.shape[:2]

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)

```



```

detection_result = self._detector.detect(mp_image)

persons = []

if detection_result.pose_landmarks:

    for landmarks in detection_result.pose_landmarks:

        person = self._extract_person(frame, landmarks, w, h)

        if person is not None:

            person["landmarks"] = landmarks

            persons.append(person)

return persons

def draw_landmarks(self, frame: np.ndarray, landmarks) -> None:

    """Draw MediaPipe pose landmarks on the frame."""

    for lm in landmarks:

        x, y = int(lm.x * frame.shape[1]), int(lm.y * frame.shape[0])

        cv2.circle(frame, (x, y), 2, (0, 255, 200), -1)

def release(self) -> None:

    """Release MediaPipe resources."""

    self._detector.close()

```

```
# — Internal Methods —————
```

```
def _extract_person(self, frame: np.ndarray, landmarks: list,
```

```
    img_w: int, img_h: int) -> dict | None:
```

```
    """Extract ROIs and features for a single detected person."""
```

```
# — Compute Head Bounding Box —————
```

```
head_bbox = self._compute_head_bbox(landmarks, img_w, img_h)
```

```
# — Compute Torso Bounding Box —————
```

```
torso_bbox = self._compute_torso_bbox(landmarks, img_w, img_h)
```

```
if head_bbox is None and torso_bbox is None:
```

```
    return None
```

```
# — Extract ROIs & Features —————
```

```
head_features = self._extract_head_features(frame, head_bbox)
```

```
torso_features = self._extract_torso_features(frame, torso_bbox)
```

```
# — Concatenate into a single feature vector —
```

```
features = np.concatenate([head_features, torso_features])
```

```
return {
```

```
    "head_bbox": head_bbox,
```

```

        "torso_bbox": torso_bbox,

        "head_features": head_features,

        "torso_features": torso_features,

        "features": features,

    }

```

# — Bounding Box Computation —————

```
def _compute_head_bbox(self, lm, w: int, h: int) -> tuple | None:
```

```
    """
```

```
    Derive the head bounding box from facial landmark positions.
```

```
    Expands the box upward to capture a potential helmet.
```

```
    """
```

```

    indices = [_NOSE, _LEFT_EYE, _RIGHT_EYE,

               _LEFT_EYE_INNER, _RIGHT_EYE_INNER,

               _LEFT_EYE_OUTER, _RIGHT_EYE_OUTER,

               _LEFT_EAR, _RIGHT_EAR]

```

```
    xs, ys = [], []
```

```
    for idx in indices:
```

```
        if lm[idx].visibility > 0.3:
```

```
            xs.append(lm[idx].x * w)
```

```
            ys.append(lm[idx].y * h)
```

```

if len(xs) < 3:

    return None

cx = np.mean(xs)

cy = np.mean(ys)

half_w = (max(xs) - min(xs)) / 2 * config.HEAD_ROI_EXPAND

half_h = (max(ys) - min(ys)) / 2 * config.HEAD_ROI_EXPAND

# Make the box taller upward (helmet extends above head)

x1 = cx - half_w * 1.3

y1 = cy - half_h * 2.5 # extend upward significantly

x2 = cx + half_w * 1.3

y2 = cy + half_h * 0.8

return (x1, y1, x2, y2)

def _compute_torso_bbox(self, lm, w: int, h: int) -> tuple | None:

    """

    Derive the torso bounding box from shoulder and hip landmarks.

    """

    indices = [_LEFT_SHOULDER, _RIGHT_SHOULDER, _LEFT_HIP, _RIGHT_HIP]

    xs, ys = [], []

```

```

for idx in indices:

    if lm[idx].visibility > 0.3:

        xs.append(lm[idx].x * w)

        ys.append(lm[idx].y * h)

if len(xs) < 3:

    return None

padding_x = (max(xs) - min(xs)) * (config.TORSO_ROI_EXPAND - 1) / 2
padding_y = (max(ys) - min(ys)) * (config.TORSO_ROI_EXPAND - 1) / 2

x1 = min(xs) - padding_x
y1 = min(ys) - padding_y
x2 = max(xs) + padding_x
y2 = max(ys) + padding_y

return (x1, y1, x2, y2)

```

# — Feature Extraction —————

```

def _extract_head_features(self, frame: np.ndarray,
                           bbox: tuple | None) -> np.ndarray:
    """

```

Extract HSV color features from the head ROI.

Features (10 total):

[0-2] color\_ratio for yellow, orange, white

[3-5] mean H, S, V

[6-8] std H, S, V

[9] dominant color ratio (max of the three color ratios)

"""

if bbox is None:

    return np.zeros(config.NUM\_HEAD\_FEATURES, dtype=np.float32)

roi = safe\_crop(frame, bbox)

if roi is None or roi.size == 0:

    return np.zeros(config.NUM\_HEAD\_FEATURES, dtype=np.float32)

roi = preprocess\_roi(roi)

hsv = cv2.cvtColor(roi, cv2.COLOR\_BGR2HSV)

total\_pixels = hsv.shape[0] \* hsv.shape[1]

if total\_pixels == 0:

    return np.zeros(config.NUM\_HEAD\_FEATURES, dtype=np.float32)

# Color ratios

```

ratios = []

for name in ["yellow", "orange", "white"]:

    lo, hi = config.HELMET_HSV_RANGES[name]

    mask = cv2.inRange(hsv, lo, hi)

    ratios.append(np.count_nonzero(mask) / total_pixels)


# HSV statistics

mean_hsv = hsv.mean(axis=(0, 1))

std_hsv = hsv.std(axis=(0, 1))


# Dominant color ratio

dominant = max(ratios)


features = np.array(

    ratios + mean_hsv.tolist() + std_hsv.tolist() + [dominant],

    dtype=np.float32

)

return features


def _extract_torso_features(self, frame: np.ndarray,

                           bbox: tuple | None) -> np.ndarray:

    """

    Extract HSV color features from the torso ROI.

```

Features (11 total):

[0-3] color\_ratio for yellow, green, orange, fluorescent

[4-6] mean H, S, V

[7-9] std H, S, V

[10] dominant color ratio

"""

if bbox is None:

    return np.zeros(config.NUM\_TORSO\_FEATURES, dtype=np.float32)

roi = safe\_crop(frame, bbox)

if roi is None or roi.size == 0:

    return np.zeros(config.NUM\_TORSO\_FEATURES, dtype=np.float32)

roi = preprocess\_roi(roi)

hsv = cv2.cvtColor(roi, cv2.COLOR\_BGR2HSV)

total\_pixels = hsv.shape[0] \* hsv.shape[1]

if total\_pixels == 0:

    return np.zeros(config.NUM\_TORSO\_FEATURES, dtype=np.float32)

# Color ratios

ratios = []



```

for name in ["yellow", "green", "orange", "fluorescent"]:

    lo, hi = config.VEST_HSV_RANGES[name]

    mask = cv2.inRange(hsv, lo, hi)

    ratios.append(np.count_nonzero(mask) / total_pixels)


# HSV statistics

mean_hsv = hsv.mean(axis=(0, 1))

std_hsv = hsv.std(axis=(0, 1))


# Dominant color ratio

dominant = max(ratios)


features = np.array(

    ratios + mean_hsv.tolist() + std_hsv.tolist() + [dominant],

    dtype=np.float32

)

return features

```

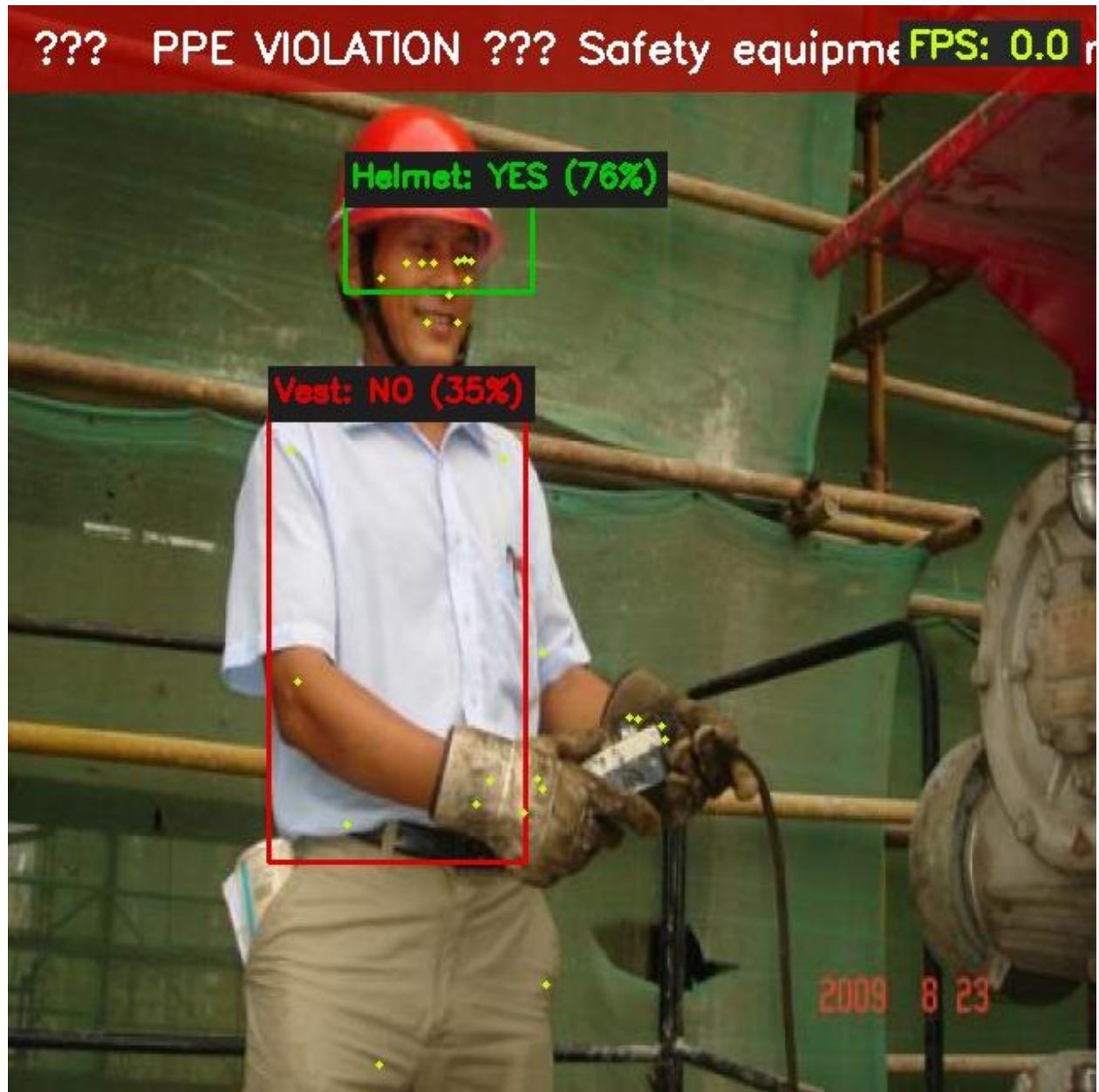
## Appendix B: Sample Output Screenshots



??? PPE VIOLATION ??? Safety equipment FPS: 0.0 r









## Appendix C: Dependencies and Installation

| Library       | Version     | Purpose                                            |
|---------------|-------------|----------------------------------------------------|
| opencv-python | $\geq 4.8$  | Frame capture, colour space conversion, annotation |
| mediapipe     | $\geq 0.10$ | Pose landmark detection (PoseLandmarker)           |
| numpy         | $\geq 1.24$ | Numerical array operations                         |
| pandas        | $\geq 2.0$  | Feature CSV I/O                                    |
| scikit-learn  | $\geq 1.3$  | Random Forest, StandardScaler, metrics             |
| joblib        | $\geq 1.3$  | Model serialisation (pkl files)                    |
| Pillow        | $\geq 10.0$ | Image I/O for static image inference mode          |

Table 6: System Dependencies